Sensores y librerías sobre STM32

Laboratorio 3 | Sistemas Electrónicos de Instrumentación

Pedro Villar | 13 de noviembre de 2026

Resolví los cuatro desafíos sobre una **STM32F4Discovery**, con un HC-SR04 y un LM35 cableados en protoboard y el acelerómetro LIS3DSH que ya viene en la placa. Acá va el lazo principal de cada desafío con sus callbacks; el proyecto completo (CubeMX + CMake, un archivo por desafío y la librería `lm35` propia) está en el repositorio.

Desafío 1: HC-SR04, librerías de terceros y actuadores

```
while (1) {
  HAL_TIM_IC_Stop_IT(sensor_distancia.htim, sensor_distancia.IC_Channel);
  hcsr04_start_measurement(&sensor_distancia);  // dispara y vuelve enseguida
  HAL_Delay(100);                               // el eco tarda ~38 ms como mucho

  if (hcsr04_is_data_ready(&sensor_distancia)) {
    float distancia = hcsr04_get_distance_cm(&sensor_distancia);
    // apagar los tres LEDs y encender solo el de la zona actual
    if (distancia > 50.0f)
      HAL_GPIO_WritePin(LED_VERDE_GPIO_Port, LED_VERDE_Pin, GPIO_PIN_SET);
    else if (distancia >= 20.0f)
      HAL_GPIO_WritePin(LED_NARANJA_GPIO_Port, LED_NARANJA_Pin, GPIO_PIN_SET);
    else
      HAL_GPIO_WritePin(LED_ROJO_GPIO_Port, LED_ROJO_Pin, GPIO_PIN_SET);
  }
}

void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim) {
  hcsr04_tim_ic_callback(&sensor_distancia, htim);  // los flancos los procesa la librería
}
```

La librería de la cátedra hace el trabajo fino: genera el pulso de trigger y mide el ancho del eco con el timer en Input Capture. Desde el main solo se dispara la medición, se espera y se pregunta si hay dato. El `Stop_IT` del principio evita que un eco perdido deje trabado el arranque siguiente. El callback no interpreta nada: le pasa los flancos a la librería, que es la que sabe qué hacer con ellos.

Desafío 2: LM35, modularización y multi-instancia

```
LM35_Init(&lm35_1, &hadc1, ADC_CHANNEL_11);   // dos instancias lógicas
LM35_Init(&lm35_2, &hadc1, ADC_CHANNEL_11);   // sobre el mismo canal físico

while (1) {
  // Fase A: lectura bloqueante de la instancia 1
  temp_bloqueante = LM35_Read_Blocking(&lm35_1);

  // Fase B: lectura no bloqueante de la instancia 2
  LM35_Start_Conversion(&lm35_2);
  float temperatura;
  while (!LM35_Get_Temperature(&lm35_2, &temperatura));  // false = sigue convirtiendo
  temp_no_bloqueante = temperatura;
}
```

La gracia está en la librería propia: cada instancia es solo una estructura con el ADC y el canal que usa, así que varias conviven sin pisarse y sin tocar el hardware a bajo nivel (de eso se encarga CubeMX). La fase A espera la conversión adentro de la función; la fase B la dispara y deja preguntar después, que es justo lo que necesita el desafío 4. Las dos temperaturas se verifican en Live Expressions con el debugger.

Desafío 3: el acelerómetro y el problema del bloqueo

```
while (1) {
  int8_t x = (int8_t)accel_leer_registro(LIS3DSH_REG_OUT_X_H);
  int8_t y = (int8_t)accel_leer_registro(LIS3DSH_REG_OUT_Y_H);
  ActualizarLedsInclinacion(x, y);              // naranja, verde y rojo

#if HEARTBEAT_BLOQUEANTE
  HAL_GPIO_TogglePin(LED_AZUL_GPIO_Port, LED_AZUL_Pin);  // el problema
  HAL_Delay(500);
#else
```

```
  if (HAL_GetTick() - millisAnterior >= 500) {          // la solución
    millisAnterior = HAL_GetTick();
    HAL_GPIO_TogglePin(LED_AZUL_GPIO_Port, LED_AZUL_Pin);
  }
#endif
}
```

Con `HAL_Delay` el azul parpadea bien, pero la inclinación se revisa solo dos veces por segundo y los LEDs responden a los tirones. Con `HAL_GetTick` el loop gira libre: el azul parpadea igual y la inclinación responde al instante. El azul queda reservado para el latido, así que la inclinación usa los otros tres LEDs. Dato de la práctica: los ejes del chip vienen rotados 90 grados respecto de los LEDs de la placa, así que el mapeo de signos salió de probar.

Desafío 4: el super-loop y la integración total

```
while (1) {
  uint32_t ahora = HAL_GetTick();

  // Tarea 1: acelerómetro, en cada vuelta
  int8_t x = (int8_t)accel_leer_registro(LIS3DSH_REG_OUT_X_H);
  int8_t y = (int8_t)accel_leer_registro(LIS3DSH_REG_OUT_Y_H);
  ActualizarLedsInclinacion(x, y, alarma);

  // Tarea 2: temperatura cada 1000 ms, fase B de la librería lm35
  if (ahora - ultimo_temperatura >= 1000) {
    ultimo_temperatura = ahora;
    LM35_Start_Conversion(&sensor_temperatura);
    conversion_en_curso = 1;
  }
  if (conversion_en_curso) {
    float temp_nueva;
    if (LM35_Get_Temperature(&sensor_temperatura, &temp_nueva)) {
      temperatura = temp_nueva;                // valor consolidado del segundo
      conversion_en_curso = 0;
    }
  }

  // Tarea 3: distancia cada 100 ms
  if (ahora - ultimo_distancia >= 100) {
    ultimo_distancia = ahora;
    if (hcsr04_is_data_ready(&sensor_distancia))
      distancia = hcsr04_get_distance_cm(&sensor_distancia);
    HAL_TIM_IC_Stop_IT(sensor_distancia.htim, sensor_distancia.IC_Channel);
    hcsr04_start_measurement(&sensor_distancia);
  }

  // Tarea 4: alarma por obstáculo a menos de 15 cm
  alarma = (distancia < 15.0f);
  if (alarma && ahora - ultimo_parpadeo >= 100) {   // 5 Hz
    ultimo_parpadeo = ahora;
    HAL_GPIO_TogglePin(LED_ROJO_GPIO_Port, LED_ROJO_Pin);
  }
}
```

No hay un solo `HAL_Delay`: cada tarea guarda la última vez que corrió y compara contra el tick actual, así ninguna frena a las demás. El acelerómetro se lee siempre, la temperatura usa la fase B de `lm35` y la distancia se recoge y se vuelve a disparar cada 100 ms. La alarma es solo una bandera: cuando hay obstáculo cerca, el rojo deja de obedecer al acelerómetro y parpadea a 5 Hz; al despejarse, vuelve solo a indicar inclinación.

Conclusiones

El laboratorio junta las tres formas de entrar datos al micro: un sensor digital que se mide con un timer (HC-SR04), uno analógico que pasa por el ADC (LM35) y uno que habla por un bus serie (el acelerómetro por SPI). Del lado del software, la lección fue separar capas: la librería resuelve el sensor y el main decide qué hacer con el dato. Escribir `lm35` con instancias y fases me ayudó a entender la librería de la cátedra, que sigue la misma idea. Y el desafío 4 cierra lo que venía del laboratorio 2: en el super-loop final cada tarea decide si le toca correr mirando el reloj, y por eso tres sensores y una alarma conviven en un mismo hilo sin molestarse.